

iSENSOR UART Debugger

인수인계서

PC-side UART 시리얼 디버거 및 AI 분석 도구

작성일: 2026-05-26

버전: v1.5.29

언어: Python 3.12 | GUI: PyQt6

대상 디바이스: ESP32-C3 PIR 센서

1. 프로젝트 개요

iSENSOR UART Debugger 는 ESP32-C3 SUPER MINI 기반 PIR 재실감지 센서
펌웨어(iSENSOR_PIR_ESP32_C3_MINI_FW)와 UART(576000 bps)로 통신하는 PC-side 실시간
모니터링/분석 도구입니다. PyQt6 기반 GUI 로 ADC 파형, FFT 스펙트럼, SVM/MLP 분류 결과를
실시간 시각화하며, AI 학습 데이터 수집·훈련·내보내기 워크플로를 통합 제공합니다.

1.1 주요 기능

기능	설명
실시간 시리얼 모니터링	UART 576 kbps, ADC 100 Hz 파형, FFT 스펙트럼 그래프
ESP32 파라미터 제어	TP1/TP2, LED 밝기/시간, FFT Stride, Sleep 시간 등 실시간 명령 송신
SVM 분류 시각화	scikit-learn RBF-SVM, PCA 2D 경계면 실시간 표시
MLP 학습/추론 (PyTorch)	GPU(CUDA 12.4) 지원, Float32·Int8 양자화 내보내기 (.h/.c)
웹캠 사람 감지 통합	YOLOv8 / MediaPipe / HOG 백엔드 선택, 학습 데이터 자동 레이블링
학습 데이터 수집	FFT 특징 21 개를 CSV 로 저장, stride/interval/cam_label 포함
BLE 통신 (bleak)	무선 디버깅 지원 (ble_worker.py)
Model Explorer GUI	AI/mlp/models/model_explorer.py, 필터·비교·곡선 분석

1.2 버전 정보

버전	날짜	주요 변경 사항
v1.5.29	2026-05-22	SVM/MLP 갱신 주기 최적화(UI 멈춤 제거), 스냅샷 상주 워커
v1.5.28	2026-05-15	MLP export 모델 선택 UI 추가
v1.5.27	2026-05-15	Int8 export 아키텍처 자동 추론, 캘리브레이션 이상치 대응
v1.5.26	2026-05-15	카메라 백엔드 선택 UI (AUTO/YOLO/MediaPipe/HOG)
v1.5.25	2026-05-14	CSV cam_label 편집기 (csv_label_editor.py)
v1.5.24	2026-05-14	MLP cam 컬럼 특징 포함 버그 수정
v1.5.21	2026-05-14	카메라 라벨링 GUI 통합 (webcam_worker.py)
v1.5.16	2026-05-13	vision 패키지 신규 (YOLOv8/MediaPipe/HOG)
v1.5.14	2026-05-13	csv_layout.py 컬럼 레이아웃 중앙화
v1.5.11	2026-05-13	Weight Decay(L2), stride/interval CSV 컬럼
v1.5.0	2026-05-11	MLP 파일 명명 규칙 통일, 시각화 모델 선택
v1.4.0	2026-05-10	MLP 학습 중단 버튼, 예폭 상한 100,000
v1.3.0	2026-05-09	Dropout SpinBox, 자동 스케일 토글
v1.2.0	2026-05-07	파일 구조 리팩토링, 예폭 소요 시간 출력
v1.1.0	2026-05-07	LR Scheduler Patience/Factor GUI
v1.0.x	초기	PyQt6 기반 UART 디버거 기초 구현

2. 개발 환경

2.1 시스템 요구사항

항목	요구사항	비고
OS	Windows 10/11 (64-bit)	macOS/Linux 실행 가능, 단 setup.ps1 비지원
Python	3.12 권장 (GPU 학습 시 필수)	PyTorch CUDA 빌드는 Python $\leq$ 3.12 만 지원
GPU	NVIDIA GTX 1660 이상 (선택)	CUDA 12.4 드라이버, GPU 없으면 CPU 모드
RAM	8 GB 이상 권장	MLP 학습 시 최소 4 GB
Python 버전	3.12 (GPU 사용 시)	버전 확인: python --version

2.2 주요 패키지 의존성

패키지	버전 요건	용도
PyQt6	$\geq$ 6.4.0	GUI 프레임워크 (메인 윈도우, 위젯)
pyqtgraph	$\geq$ 0.13.0	실시간 그래프 (ADC, FFT, SVM)
pyserial	$\geq$ 3.5	UART 시리얼 통신
bleak	$\geq$ 0.21.0	BLE 무선 통신
numpy	$\geq$ 1.24.0	수치 연산, FFT
scikit-learn	$\geq$ 1.3.0	SVM 분류기
torch	$\geq$ 2.0.0	MLP 신경망 학습 (GPU: cu124 빌드 권장)
matplotlib	$\geq$ 3.7.0	학습 곡선 시각화
pandas	$\geq$ 1.5.0	CSV 데이터 처리
opencv-python	$\geq$ 4.8.0	웹캠 영상 처리, HOG 사람 감지
ultralytics	$\geq$ 8.0.0	YOLOv8 사람 감지 (yolov8n.pt 자동 다운로드)
mediapipe	$\geq$ 0.10.0	MediaPipe Pose (0.10+는 solutions API 제거됨)

⚠ mediapipe 0.10+는 solutions API 가 제거되어 MediaPipe 백엔드가 실질적으로 비활성화됩니다. 사람 감지는 YOLO 또는 HOG 백엔드 사용을 권장합니다.

2.3 설치 방법 — 방법 A (자동, 권장)

setup.ps1 스크립트를 실행하면 GPU 유무 자동 감지 후 적합한 PyTorch 빌드를 설치합니다.

```
# PowerShell 에서 실행
.\setup.ps1
```

자동 설치 순서:

1. Python 3.12 미설치 시 winget 으로 자동 설치
2. .venv 가상환경 생성 (Python 3.12)
3. NVIDIA GPU 감지 → GPU 빌드 / 미감지 → CPU 빌드 torch 설치
4. requirements.txt 나머지 의존성 설치

2.4 설치 방법 — 방법 B (수동)

```
# 1. Python 3.12 설치 (GPU 사용 시 필수)
winget install Python.Python.3.12

# 2. 가상환경 생성 (GPU)
py -3.12 -m venv .venv

# 3. 가상환경 활성화
.venv\Scripts\Activate.ps1

# 4-A. GPU(CUDA 12.4) 의존성 설치
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu124
pip install -r requirements.txt

# 4-B. CPU 전용
pip install -r requirements.txt

# 5. 실행
python debugger_start.py
```

2.5 GPU 환경 확인

GUI 실행 후 MLP Training 패널 하단의 GPU 상태 표시로 확인합니다.

표시	의미
● GPU: CUDA (GTX 1660)	GPU 학습 활성화
● CPU only	CPU 전용 모드 (Python ≤ 3.12 환경에서 cu124 빌드 재설치 필요)

3. 소스코드 구조

3.1 디렉터리 트리

```
iSENSOR_UART_DEBUGGER/
├─ debugger_start.py      # ★ 메인 진입점 (PyQt6 MainWindow)
├─ config.py              # 전역 상수 (창 크기, 색상, UART 프레임 상수, CMD 코드)
├─ endian_converter.py    # 빅/리틀 엔디안 변환 유틸
├─ ble_worker.py          # BLE 통신 워커 (bleak)
├─ setup.ps1              # 자동 설치 스크립트 (PowerShell)
├─ requirements.txt       # pip 의존성
├─
├─ uart_protocol/         # UART 프로토콜 패키지
│   └─ uart_protocol_config.py # 프레임 구조·크기 상수, UartDataType, UartCommandType
│   └─ uart_receive_parser.py  # STX~ETX 프레임 파서 (상태 머신)
│   └─ data_models.py         # 수신 데이터 모델 (dataclass)
│   └─ data_parser.py        # 페이로드 → Python 객체 변환
│   └─ command_sender.py     # PC→ESP32 명령 프레임 생성·송신
│   └─ checksum.py           # Sum 체크섬 계산/검증 (16-bit)
│   └─
├─ fft/
│   └─ __init__.py          # numpy rfft 기반 FFT 모듈 (PC 측 재계산)
│   └─
├─ AI/                    # AI/ML 패키지
│   └─ csv_layout.py        # CSV 컬럼 레이아웃 SSOT (21 특징, 256 ADC, 129 FFT)
│   └─ training_data_collector.py # FFT 샘플 → CSV 저장
│   └─ svm/
│       └─ svm.py           # RBF-SVM 학습·추론 (scikit-learn)
│   └─ mlp/
│       └─ nn_mlp.py        # OccupancyMLP 학습·저장 (PyTorch)
│       └─ pc_feature_extractor.py # ADC → 21 개 FFT 특징 추출
│       └─ export/
│           └─ export_float32.py # Float32 .h/.c 내보내기
│           └─ export_int8.py    # Int8 양자화 내보내기
│       └─ models/
│           └─ model_explorer.py # PyQt6 모델 비교 탐색기
│       └─ results/
│           └─ visualize.py    # 학습 결과 대시보드 PNG 생성
│       └─
├─ vision/                # 웹캠 사람 감지 패키지
│   └─ webcam_person_detector.py # YOLO/MediaPipe/HOG 백엔드 통합
```

```

|   ├── webcam_worker.py      # PyQt6 QThread 래퍼
|   └── test_webcam_detector.py # 독립 성능 검증 스크립트
|
├── data_csv/                  # 학습 데이터 CSV 저장 폴더 (자동 생성)
|   └── snapshots/             # 웹캠 스냅샷 JPEG (타임스탬프 파일명)
|
└── docs/                      # 문서
    └── generate_handover_doc.py # 이 스크립트

```

3.2 모듈 역할 요약

파일/모듈	역할
debugger_start.py	PyQt6 메인 윈도우. 모든 탭/패널 생성, 이벤트 핸들러, 워커 스레드 관리
config.py	윈도우·색상·UART 상수(STX/ETX/CMD 코드/필드 크기) 중앙 정의
endian_converter.py	Big-Endian/Little-Endian uint16/32/64 변환 함수
ble_worker.py	bleak 기반 BLE GATT 비동기 수신 QThread
uart_protocol/uart_protocol_config.py	프레임 구조(헤더·페이로드·체크섬 크기), UartDataType/UartCommandType 열거형
uart_protocol/uart_receive_parser.py	STX 탐색 → 헤더 파싱 → 페이로드 누적 → 체크섬 검증 상태 머신
uart_protocol/checksum.py	Sum 체크섬 16-bit: calculate_checksum(), verify_checksum()
fft/_init_.py	numpy.fft.rfft 기반 PC-side FFT (DC 제거, 진폭 정규화, 피크 탐색)
AI/csv_layout.py	CSV_N_FEATURES=21, CSV_N_ADC=256, CSV_N_FFT=129 등 컬럼 인덱스 SSOT
AI/training_data_collector.py	FFT 특징 + ADC + cam_meta → CSV 행 저장 (stride/interval 태그 포함)
AI/svm/svm.py	RBF-SVM 학습(StandardScaler → SVM.fit), PCA 2D 변환, 예측
AI/mlp/nn_mlp.py	OccupancyMLP(Linear-ReLU-Dropout) 학습, ReduceLRonPlateau, history JSON
AI/mlp/pc_feature_extractor.py	ADC 256 샘플 → numpy FFT → 21 개 특징 추출 (ESP32 와 동일 파이프라인)
AI/mlp/export/export_float32.py	학습된 .pt → Float32 C 배열 .h/.c 내보내기 (ESP32 임베딩용)
AI/mlp/export/export_int8.py	학습된 .pt → Int8 양자화 C 배열 .h/.c 내보내기 (정밀도 최적화)
AI/mlp/models/model_explorer.py	PyQt6 모델 탐색기: 필터·지표 비교·Val/Train 곡선 오버레이
vision/webcam_person_detector.py	YOLO v8/v9/v10/v11, MediaPipe Pose, OpenCV HOG 감지 통합
vision/webcam_worker.py	WebcamWorker(QThread): 프레임 시그널, 시간적 앙상블, is_fresh()

4. 실행 방법

4.1 메인 GUI 실행

```
# 가상환경 활성화 후
python debugger_start.py
```

4.2 독립 실행 도구

스크립트	목적	실행 명령
vision/test_webcam_detector.py	웹캠 감지 성능 단독 검증	python vision/test_webcam_detector.py
AI/mlp/export/export_float32.py	Float32 모델 ESP32 C 코드 내보내기	python AI/mlp/export/export_float32.py
AI/mlp/export/export_int8.py	Int8 양자화 모델 내보내기	python AI/mlp/export/export_int8.py
AI/mlp/models/model_explorer.py	모델 비교 탐색기 GUI	python AI/mlp/models/model_explorer.py
AI/mlp/results/visualize.py	학습 결과 대시보드 PNG 생성	python AI/mlp/results/visualize.py
csv_label_editor.py	cam_label 수동 편집기	python csv_label_editor.py

5. UART 프로토콜

5.1 통신 설정

항목	값
Baud Rate	576,000 bps (ESP32 sdkconfig.defaults 에서 설정)
데이터 비트	8 bit
정지 비트	1 bit
패리티	None
방향	양방향 (ESP32 → PC: 데이터 스트림 / PC → ESP32: 명령 송신)

5.2 수신 프레임 구조 (ESP32 → PC)

STX(3 bytes)	DATA_TYPE(1)	DATA_LENGTH(2)	PAYLOAD(N)	CHECKSUM(2)	ETX(3 bytes)
0xAA 0x55 0xCC					0xDD 0x55
0xAA					

헤더 총 6 bytes = STX(3) + DATA_TYPE(1) + DATA_LENGTH(2)

CHECKSUM : STX ~ PAYLOAD 전체 바이트 합의 하위 16-bit (Big Endian)

5.3 DATA_TYPE 정의

값	이름	페이로드 크기	설명
0	RAW_VALUE	가변	ADC 단일 원시값
1	ADC_BUFFER	512 B	256 샘플 × 2 bytes (uint16 Big Endian)
9	SETTINGS	47 B	현재 ESP32 설정값 전체
11	PROFILING	44 B	11 개 처리 시간 × 4 bytes (uint32 Big Endian) (단위: μs)
12	FFT	516 B	129 진폭 × 4 bytes (uint32 Big Endian)
13	FFT_FEATURES	84 B	21 개 특징 × 4 bytes (float/int32/uint32 Big Endian)
14	MLP_RESULT	16 B	Float label(int32) + Float prob + Int8 label + Int8 prob

5.4 SETTINGS 페이로드 구조 (47 bytes)

필드	타입	크기	설명
TP1	uint16	2 B	재실 감지 임계값 (에너지 기준)
TP1_RECHECK	uint16	2 B	재실 재확인 임계값
TP2	uint64	8 B	연속 감지 카운트 임계값
LED_MAX_PER	uint8	1 B	LED 최대 밝기 (%)
LED_MIN_PER	uint8	1 B	LED 최소 밝기 (%)
LED_DIM_PER	uint8	1 B	LED 디밍 밝기 (%)

LED_WORK_MS	uint32	4 B	LED 점등 유지 시간 (ms)
LED_STEP_MS	uint32	4 B	LED 디밍 스텝 시간 (ms)
LED_DELAY_MS	uint32	4 B	LED 디밍 딜레이 시간 (ms)
OCCU_CHK_TIMEOUT	uint64	8 B	재실 확인 타임아웃 (ms)
SLEEP_TIME	uint64	8 B	딤슬립 시간 (ms)
OCCUPANCY_STATUS	uint8	1 B	현재 재실 상태 (0=부재, 1=재실)
PIR_STATUS	uint8	1 B	PIR 출력 상태
FFT_STRIDE	uint16	2 B	FFT 계산 주기 (샘플 단위)

5.5 PROFILING 페이로드 (44 bytes = 11 × uint32 Big Endian)

인덱스	필드명	단위
0	adc_reading_time_us	μs
1	adc_read_buffer_latency_time_us	μs
2	adc_processing_time_us	μs
3	adc_buffer_insert_time_us	μs
4	fft_process_time_us	μs
5	fft_features_process_time_us	μs
6	fft_loop_a_time_us	μs
7	fft_loop_b_time_us	μs
8	fft_loop_c_time_us	μs
9	float32_mlp_infer_time_us	μs
10	int8_mlp_infer_time_us	μs

5.6 PC → ESP32 명령 프레임

명령 프레임은 동일한 STX/ETX 구조를 사용하며, DATA_TYPE 필드에 CMD 코드를 기입합니다.

CMD 코드	이름	페이로드	설명
0x10	CMD_SET_TP1	uint16 (2 B)	TP1 임계값 설정
0x11	CMD_SET_TP2	uint64 (8 B)	TP2 카운트 설정
0x12	CMD_SET_TP1_RECHECK	uint16 (2 B)	TP1 재확인 임계값
0x13	CMD_SET_FFT_STRIDE	uint16 LE (2 B)	FFT Stride 설정
0x14	CMD_SET_LED_MAX_PER	uint8 (1 B)	LED 최대 밝기 (%)
0x15	CMD_SET_LED_MIN_PER	uint8 (1 B)	LED 최소 밝기 (%)
0x16	CMD_SET_LED_DIM_PER	uint8 (1 B)	LED 디밍 밝기 (%)
0x17	CMD_SET_LED_WORK_MS	uint32 LE (4 B)	LED 점등 유지 (ms)
0x18	CMD_SET_LED_STEP_MS	uint32 LE (4 B)	LED 디밍 스텝 (ms)
0x19	CMD_SET_LED_DELAY_MS	uint32 LE (4 B)	LED 디밍 딜레이 (ms)
0x1A	CMD_SET_OCCU_TIMEOUT	uint32 LE (4 B)	재실 확인 타임아웃 (초)
0x1B	CMD_SET_SLEEP_TIME	uint32 LE (4 B)	딤슬립 시간 (초)
0x1C	CMD_SET_LED_ONOFF	uint8 (1 B)	LED ON(1)/OFF(0)
0x1D	CMD_SET_MLP_FLOAT_ENABLE	uint8 (1 B)	Float32 MLP 추론 ON/OFF
0x1E	CMD_SET_MLP_INT8_ENABLE	uint8 (1 B)	Int8 MLP 추론 ON/OFF
0x20	CMD_GET_SETTINGS	없음	현재 설정값 요청

0x30	CMD_SAVE_NVS	없음	설정을 NVS 에 저장
0xF0	CMD_RESET	없음	ESP32 소프트 리셋

6. AI/ML 파이프라인

6.1 CSV 컬럼 레이아웃 (csv_layout.py)

모든 AI 모듈이 이 파일의 상수를 참조하여 CSV 구조를 공유합니다.

상수	값	설명
CSV_N_FEATURES	21	FFT 특징 수 (컬럼 0~20)
CSV_N_ADC	256	ADC 원시 버퍼 (컬럼 21~276)
CSV_N_FFT	129	FFT 진폭 버퍼 (컬럼 277~405)
CSV_N_META	2	stride, interval (컬럼 406~407)
label 컬럼	-	CSV 마지막 열 (0=배경, 1=사람)
cam_label	-	웹캠 자동 레이블 (-1=미지정, 0=BG, 1=Human)
cam_hm_conf	-	사람 확신도 (0.0~1.0)
cam_bg_conf	-	배경 확신도 (0.0~1.0)
timestamp	-	UNIX 타임스탬프 (스냅샷 매핑)

6.2 FFT 특징 21 개

인덱스	특징명	설명
0	f_avg_energy	평균 에너지 (0Hz 제외)
1	f_std_energy	에너지 표준편차
2	f_peak_energy	최대 에너지 (피크 진폭)
3	f_peak_freq	피크 주파수 (Hz)
4	f_spectral_centroid	스펙트럼 무게중심
5	f_spectral_spread	스펙트럼 분산
6	f_spectral_skewness	스펙트럼 비대칭도
7	f_spectral_kurtosis	스펙트럼 첨도
8	f_band0_energy	주파수 밴드 0 에너지
9	f_band1_energy	주파수 밴드 1 에너지
10	f_band2_energy	주파수 밴드 2 에너지
11	f_band3_energy	주파수 밴드 3 에너지
12	f_band4_energy	주파수 밴드 4 에너지
13	f_band_ratio	밴드 에너지 비율
14	f_spectral_rolloff	스펙트럼 롤오프 주파수
15	f_spectral_entropy	스펙트럼 엔트로피
16	f_sfm	스펙트럼 평탄도 (SFM)
17	f_harmonic_ratio	고조파 비율
18	i_peak_count	피크 개수 (int32)
19	ui32_avg_energy	평균 에너지 (uint32, ESP32 sc16 FFT 결과)
20	ui32_peak_energy	피크 에너지 (uint32, ESP32 sc16 FFT 결과)

6.3 SVM 학습 흐름

```
CSV 로드 (data_csv/svm_data_*.csv)
↓
특징 21 개 + 레이블 분리
↓
StandardScaler.fit_transform()
↓
SVC(kernel='rbf', C=1.0, gamma='scale').fit()
↓
PCA(n_components=2) 변환 → 경계면 시각화
```

6.4 MLP 학습 흐름 (nn_mlp.py)

```
CSV 로드 (stride/interval/cam 필터 적용)
↓
StandardScaler.fit_transform()
↓
OccupancyMLP(21 → HIDDEN_LAYERS → 2) 생성
- 기본 레이어: [64, 32] (또는 [128,64,32], [256,128,64,32] 선택 가능)
- Dropout: 0.3 (기본)
↓
Adam(lr=5e-4) + ReduceLROnPlateau(patience=10, factor=0.5)
↓
CrossEntropyLoss, 에폭별 train/val loss 기록 → _history.json
↓
best val_acc 시점 모델 저장 → models/MLP_{n}_{tag}/*.pt + *_scaler.pkl
```

6.5 모델 내보내기 (ESP32 임베딩)

스크립트	출력	설명
export_float32.py	mlp_float32.h, mlp_float32.c	Float32 가중치 C 배열, ESP32 mlp_float32 모듈에 직접 복사
export_int8.py	mlp_int8.h, mlp_int8.c	Int8 양자화 가중치, 99% percentile scale 로 이상치 내성 확보

7. 주요 설정 상수 (config.py)

상수	값	설명
WINDOW_TITLE	iSENSOR PIR SENSOR DEBUG	메인 윈도우 제목
WINDOW_WIDTH/HEIGHT	2500 / 550	기본 윈도우 크기 (px)
BACKGROUND_COLOR	#003D83	그래프 배경 (네이비 블루)
FFT_SAMPLING_RATE	100	ADC 샘플링 속도 (Hz)
WINDOW_SIZE	256	FFT/ADC 슬라이딩 윈도우 크기 (ESP32 project_top.h 와 동일 유지)
MAX_LOG_LINES	500	로그 창 최대 표시 줄
UART_RECEIVE_START_PATTERN	0xAA 0x55 0xCC (3 bytes)	UART 수신 프레임 시작 패턴
UART_RECEIVE_END_PATTERN	0xDD 0x55 0xAA (3 bytes)	UART 수신 프레임 종료 패턴

⚠ WINDOW_SIZE(256)는 ESP32 펌웨어의 project_top.h WINDOW_SIZE 와 반드시 동일해야 합니다. 두 값이 다르면 FFT 결과 매핑이 어긋납니다.

8. 웹캠 사람 감지 (vision 패키지)

8.1 백엔드 구조

백엔드	우선순위	장점	단점/비고
YOLO v8~v11	1 (AUTO 기본)	부분 가림 강인, BG 확신도 probe 지원	첫 실행 시 모델 자동 다운로드 (~6 MB)
MediaPipe	2	관절 33 개 스켈레톤	0.10+ solutions API 제거됨 → 실질적 비활성
HOG+SVM	3	의존성 없음 (OpenCV 내장)	전신 기준, 상반신/부분 가림 취약

8.2 시간적 앙상블 (Temporal Smoothing)

최근 N 프레임 중 M 개 이상 Human 판정 시 최종 Human 으로 확정합니다.

```
smooth_window = 10    # 최근 10 프레임
smooth_thresh = 3      # 3 개 이상 Human → 최종 Human
```

8.3 YOLO BG 확신도 계산

YOLO 는 BG 판정 시 자체 confidence=0 이므로, low-conf probe(conf=0.01)로 raw score 를 수집하여 BG 확신도 = $1.0 - \max_raw_person_score$ 로 계산합니다.

9. 학습 데이터 수집 워크플로

9.1 수동 레이블링 수집 절차

- ① GUI → SVM 수집 패널 → 레이블 소스: '수동' 선택
- ② label 라디오 버튼으로 Background(0) / Occupancy(1) 선택
- ③ 자동 저장 간격(stride), FFT 갱신 횟수(interval) 설정
- ④ '저장 시작' 클릭 → data_csv/svm_data_{timestamp}.csv 에 축적

9.2 카메라 자동 레이블링 수집 절차

- ① 카메라 ON 버튼 클릭 → 웹캠 감지 시작
- ② 레이블 소스: '카메라' 선택
- ③ FFT 샘플 저장 시 is_fresh() 확인 후 cam_meta 포함 자동 저장
- ④ 스냅샷 저장 체크박스 ON → data_csv/snapshots/frame_{ts:.3f}.jpg 동시 저장
- ⑤ csv_label_editor.py 로 cam_label 시각적 검토·편집

9.3 CSV 파일명 규칙

```
data_csv/svm_data_{YYYYMMDD}_{HHMMSS}.csv  
data_csv/snapshots/frame_{unix_timestamp:.3f}.jpg
```

10. 주의사항 및 인수 체크리스트

10.1 환경 관련 주의사항

항목	내용
Python 버전	GPU 학습 시 반드시 Python 3.12. PyTorch CUDA 공식 릴 최대 지원 버전
WINDOW_SIZE	config.py(PC)와 project_top.h(ESP32)의 WINDOW_SIZE 를 반드시 256 으로 동일하게 유지
Baud Rate	576,000 bps — 표준 Baud Rate 아님. pyserial 에서 정수값으로 직접 지정
FFT_STRIDE	ESP32 펌웨어에서 CMD_SET_FFT_STRIDE 명령으로 변경 가능, PC GUI 와 동기화 필요
mediapipe	0.10+ 환경에서 MediaPipe 백엔드는 비활성 상태. YOLO/HOG 사용 권장
YOLOv8 모델	yolov8n.pt 첫 실행 시 인터넷에서 자동 다운로드 (~6 MB). 오프라인 환경 불가
Int8 export	캘리브레이션 데이터(CSV) 필요. percentile 99% scale 방식으로 이상치 내성
학습 데이터 CSV	data_csv/ 폴더는 .gitignore 에 포함 권장 (용량 클 수 있음). USB 복사 시 포함 여부 확인

10.2 인수 체크리스트

확인	항목
☐	Python 3.12 환경 구성 및 .venv 생성 확인
☐	GPU 사용 시 CUDA 12.4 드라이버 설치 및 torch CUDA 빌드 확인
☐	python debugger_start.py 실행 → GUI 정상 동작 확인
☐	COM 포트 선택 및 ESP32 연결 (576,000 bps) 확인
☐	ADC 그래프, FFT 그래프 실시간 표시 확인
☐	CMD_GET_SETTINGS 명령 → SETTINGS 수신 확인
☐	SVM 학습 데이터 CSV 수집 → SVM 학습 → PCA 경계면 표시 확인
☐	MLP 학습 → val_acc 수렴 확인 → Float32/Int8 내보내기 확인
☐	내보낸 .h/.c 파일을 ESP32 프로젝트(iSENSOR_PIR_ESP32_C3_MINI_FW/main/app_level/AI/)에 적용
☐	data_csv/ 폴더 내 학습 데이터 USB 백업 확인
☐	AI/mlp/models/ 폴더 내 학습된 모델(.pt, _scaler.pkl) 백업 확인
☐	CHANGELOG.md 최신 이력 확인 (현재 v1.5.29)

11. 관련 프로젝트 및 참고 자료

항목	내용
ESP32 펌웨어	iSENSOR_PIR_ESP32_C3_MINI_FW (동일 디렉터리 상위)
PyQt6 공식 문서	https://doc.qt.io/qtforpython-6/
PyTorch 문서	https://pytorch.org/docs/stable/
Ultralytics YOLO	https://docs.ultralytics.com/
pyqtgraph	https://pyqtgraph.readthedocs.io/
bleak (BLE)	https://bleak.readthedocs.io/
CHANGELOG.md	프로젝트 루트 CHANGELOG.md — 전체 버전 이력

AI/mlp/TUNING_GUIDE.md	MLP 하이퍼파라미터 튜닝 가이드 (과적합 진단, LR 설정, 에폭 기준)
------------------------	---